IMPORTANT **A Bayesian optimization policy scores each potential query according to their value and thus decides where we should query the objective function next (where the score is maximized). This score computed by the policy is called the acquisition score, as we are using it as a method of data acquisition.**

A good policy that assigns high scores to data points that are truly valuable to optimization will point us towards the optimum of the objective function more quickly and efficiently, while a badly designed policy might misguide our experiments and waste valuable resources.

Unfortunately, there's no perfect solution to the problem of designing a good BayesOpt policy. That is, there's no single BayesOpt policy that consistently excels across all objective functions. Different policies, as we will see in this and the following chapters, use different focuses and heuristics. Some heuristics might work well on a particular type of objective function, while others might prove effective on other functions. This means we need to expose ourselves to a wide range of BayesOpt policies and understand what they aim to do to be able to apply them to appropriate situations, and that's exactly what we will be doing in Chapters 4–6.

SIDEBAR **What is a policy?**

Each Bayesian optimization policy is a decision rule that scores data points in terms of usefulness in optimization based on a given criterion or heuristic. Different criteria and heuristics give rise to different policies, and there is no predetermined set of Bayesian optimization policies. In fact, Bayesian optimization researchers still publish papers that propose new policies. In this book, we will only learn about the most popular and commonly used policies in practice.

Let us now take a moment to implement a placeholder BayesOpt loop, which we will be using from now on to inspect various BayesOpt policies. This code is implemented in `CH04/01 - BayesOpt loop.ipynb`. The first component we need is an objective function that we'd like to optimize using BayesOpt. Here, we use the familiar one-dimensional Forrester function, which is defined to be between -5 and 5. We also compute the value of the Forrester function inside *[-5, 5]* with `xs` and `ys` as the ground truth.

```
def forrester_1d(x):   ❶
    # a modification of https://www.sfu.ca/~ssurjano/forretal08.html   ❶
    y = -((x + 1) ** 2) * torch.sin(2 * x + 2) / 5 + 1   ❶
    return y.squeeze(-1)   ❶


bound = 5   ❷

xs = torch.linspace(-bound, bound, bound * 100 + 1).unsqueeze(1)   ❷
ys = forrester_1d(xs)   ❷
```

❶ Form of the objective function, assumed to be unknown.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>